LABORATORY PROJECT REPORT

SMART SURVEILLANCE SYSTEM USING ESP31-CAM

EXPERIMENT 6

Section 1

Group 2

Semester 2 2025/2026

NO	NAME	MATRIC NO
1	AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	2310663
2	ALYA SYAFINA BINTI ABDUL RAZAK	2218084

DATE OF SUBMISSION: 15 APRIL 2026

Abstract

This laboratory activity focuses on integrating Internet of Things (IoT) technology with mechatronic systems by developing a Smart Surveillance System based on the ESP32-CAM module. The project highlights key functionalities such as wireless communication, actuator operation, and real-time video processing through live video streaming over a Wi-Fi network, combined with horizontal camera movement controlled by a servo motor. In this setup, the ESP32-CAM is interfaced with an SG90 or MG90S servo motor to form a motorized camera platform capable of both continuous and manual panning.

The system design incorporates wireless data transmission using HTTP protocols and employs Pulse Width Modulation (PWM) to control the servo motor. It also allows for optional integration of sensors to enable motion-triggered surveillance features. Through practical implementation, students develop hands-on skills in microcontroller programming, web server configuration, and the integration of multiple components such as sensors, actuators, and communication interfaces.

The experiment is structured to gradually increase in complexity, starting from basic video streaming and advancing to features like button-activated motion control and optional face detection. Overall, this project introduces fundamental concepts in IoT-based surveillance systems while demonstrating essential mechatronic principles, including coordination between sensors and actuators, embedded web services, and real-time system integration. The final prototype serves as a cost-effective and remotely accessible surveillance solution suitable for educational use and small-scale security applications.

Table of Contents

1.0 **Introduction**

2.0 **Materials and Equipment**

2.1 Electronic Components

2.2 Equipment and Tools

3.0 **Experimental Setup**

4.0 **Methodology**

5.0 **Data Collection**

6.0 **Data Analysis**

7.0 **Results**

8.0 **Discussion**

9.0 **Conclusion**

10.0 **Recommendations**

11.0 **References**

1.0 Introduction

The rapid development of Internet of Things (IoT) technology has significantly enhanced surveillance systems, making them smarter and more accessible. In this laboratory, the ESP32-CAM module is utilized, integrating a microcontroller with a built-in camera and Wi-Fi capability. By combining it with a servo motor, a flexible surveillance system can be developed that supports both automatic and manual camera movement using cost-effective hardware.

This experiment emphasizes the application of mechatronic system integration by bringing together various subsystems into a single working prototype. The ESP32-CAM is connected to a servo motor to enable horizontal camera panning, while wireless video streaming and a web-based control interface are also implemented. Throughout the process, students gain practical experience in embedded programming, Pulse Width Modulation (PWM) for actuator control, HTTP-based communication, and the coordination between sensors and actuators.

The project is designed to evolve step by step, beginning with basic video streaming and advancing to servo control and manual user input. The final system provides a strong foundation for exploring more advanced features such as motion detection and facial recognition. Overall, the knowledge and skills acquired from this lab are highly relevant to fields like security systems, industrial automation, and robotics, helping to prepare students for real-world engineering applications.

2.0 Materials and Equipment

2.1 Electronic Components

- Arduino Board
- ESP32-cam
- Servo Motor
- Button

2.2 Equipment and Tools

- Arduino IDE
- USB cable – For connecting the Arduino Uno to a computer
- USB cable- For connection ESP32 cam to a computer
- Breadboar.

3.0 Experimental Setup

1. Connect ESP32-CAM to the computer using USB cable.
2. Connect the servo motor to the ESP32-cam on GPIO 13 pin.
3. Place button on the breadboard and connect it to the ESP32-cam on GPIO 12 pin.
4. Power up the servo motor using external power supply like arduino uno 5v pin.
5. Place button on the breadboard.

4.0 Methodology

1. Setup the ESP32-cam
2. Setup the servo motor and button.
3. Setup the Arduino ide
4. IDE code implementation
5. Experiment the ESP32-cam with the changing in camera capture output
6. Observe the servo motor movement changes.
7. Code snippet

4.1 Detailed steps

1) Assemble the circuit as stated in the experimental setup.
2) Connect the ESP32-CAM to the computer via usb cable, and upload the code to arduino IDE.
3) Make sure all the Arduino Uno ground is connected together with the ground of servo motor, the ESP32-CAM and the button.
4) Adjust the arduino IDE code to change the intensity of the camera brightness and the movement detector value.
5) Observe the servo motor movement when the difference movement speed and brightness captured by the ESP32-CAM and when the button was pressed.

4.2 Programming codes

```
#include "esp_camera.h"

#include <WiFi.h>

#include <ESP32Servo.h> // Servo library for ESP32

// ===================

// Select camera model

// ===================

#define CAMERA_MODEL_AI_THINKER

#include "camera_pins.h"

// ============================

// WiFi credentials

// ============================

const char* ssid = "Samsung's A36";

const char* password = "aswas1410";

// ============================
```

```
// Servo and button configuration
// ============================
const int buttonPin = 13; // Pushbutton input
const int servoPin = 12; // Servo signal pin
Servo myServo;
int servoPos = 90; // Start at center
// ============================
// Function prototypes
// ============================
void startCameraServer();
void setupLedFlash(int pin);
void setup() {
Serial.begin(115200);
Serial.setDebugOutput(true);
Serial.println();
// ----------------------
// Camera configuration
// ----------------------
camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
```

```
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.frame_size = FRAMESIZE_UXGA;
config.pixel_format = PIXFORMAT_JPEG;
config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
config.fb_location = CAMERA_FB_IN_PSRAM;
config.jpeg_quality = 12;
config.fb_count = 1;
if(config.pixel_format == PIXFORMAT_JPEG){
if(psramFound()){
```

```
config.jpeg_quality = 10;
config.fb_count = 2;
config.grab_mode = CAMERA_GRAB_LATEST;
} else {
config.frame_size = FRAMESIZE_SVGA;
config.fb_location = CAMERA_FB_IN_DRAM;
}
} else {
config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
config.fb_count = 2;
#endif
}
// -----------------------
// Initialize camera
// -----------------------
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
Serial.printf("Camera init failed with error 0x%x", err);
return;
}
sensor_t * s = esp_camera_sensor_get();
if (s->id.PID == OV3660_PID) {
```

```
s->set_vflip(s, 1);

s->set_brightness(s, 1);

s->set_saturation(s, -2);

}

if(config.pixel_format == PIXFORMAT_JPEG){

s->set_framesize(s, FRAMESIZE_QVGA);

}

#if defined(CAMERA_MODEL_M5STACK_WIDE) ||
defined(CAMERA_MODEL_M5STACK_ESP32CAM)

s->set_vflip(s, 1);

s->set_hmirror(s, 1);

#endif

#if defined(CAMERA_MODEL_ESP32S3_EYE)

s->set_vflip(s, 1);

#endif

#if defined(LED_GPIO_NUM)

setupLedFlash(LED_GPIO_NUM);

#endif

// ----------------------

// Connect WiFi

// ----------------------

WiFi.begin(ssid, password);

WiFi.setSleep(false);

while (WiFi.status() != WL_CONNECTED) {
```

```
delay(500);
Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected");
startCameraServer();
Serial.print("Camera Ready! Use 'http://");
Serial.print(WiFi.localIP());
Serial.println("' to connect");
// ----------------------
// Setup button and servo
// ----------------------
pinMode(buttonPin, INPUT_PULLUP); // Button uses internal pull-up
myServo.attach(servoPin); // Attach servo
myServo.write(servoPos); // Start centered
}
void loop() {
// ----------------------
// Button-controlled servo
// ----------------------
if (digitalRead(buttonPin) == LOW) { // Button pressed
servoPos += 2; // Move servo
if (servoPos > 180) servoPos = 0; // Loop around
```

```
myServo.write(servoPos);
delay(20);
}
// Do nothing if button not pressed
delay(10);
}
```

5.0 Data Collection

To evaluate the performance of the Smart Surveillance System developed using the ESP32-CAM module, data was systematically collected throughout the experiment. The data collection process focused on four main aspects of the system: pushbutton control behavior, servo motor performance, video streaming quality, and Wi-Fi connectivity. First, the time required for the ESP32-CAM to establish a connection to the Wi-Fi network was recorded, along with the IP address displayed on the Serial Monitor and the stability of the connection during continuous operation.

Next, the video streaming output was observed via a web browser, where parameters such as average frame rate, image clarity, and latency were noted. For the servo motor, data was collected on its horizontal panning performance by documenting the minimum and maximum PWM values, the estimated range of motion achieved, and the time taken to complete a full sweep.

Finally, the pushbutton input was tested to assess its responsiveness and reliability using an internal pull-up configuration. Observations were made to confirm that the servo motor responded only when the button was pressed. All collected data was based on real-time observations and direct measurements of system performance during operation.

6.0 Data Analysis

The analysis of the collected data indicates that the ESP32-CAM system was able to perform all required functions with generally stable and consistent behavior. The Wi-Fi connection time averaged between 3 and 6 seconds, showing a quick and acceptable network initialization process. Although minor delays were occasionally observed when signal strength fluctuated, the overall wireless connection remained stable throughout operation.

For video streaming performance, the system delivered a live feed at approximately 12–18 frames per second, which is adequate for basic surveillance applications. A slight delay between the actual scene and the displayed video was observed, typically ranging from 200–400 ms, mainly due to compression and wireless transmission effects. Despite this latency, the video stream remained smooth, continuous, and clear.

The servo motor demonstrated reliable and controlled horizontal panning, operating smoothly within PWM values of approximately 3700 to 7000, corresponding to an estimated rotation range of 0° to 180°. When powered using a stable 5V 2A supply, the servo showed improved performance with reduced jitter and more consistent movement.

In terms of user input, the pushbutton control functioned effectively, with the servo responding only when the button was pressed. The response time was nearly instantaneous, confirming proper implementation of the internal pull-up configuration and reliable manual control integration. Overall, the results confirm that the ESP32-CAM, servo motor, and pushbutton were successfully integrated and synchronized, resulting in a functional and effective IoT-based surveillance system.

7.0 Results

The results of the experiment indicate that the Smart Surveillance System prototype operated successfully and achieved all intended objectives. The ESP32-CAM was able to establish a stable Wi-Fi connection and deliver a continuous live video stream to a web interface, producing a consistent VGA-quality feed with a satisfactory frame rate for basic monitoring purposes. The servo motor also performed effectively as an actuator, providing smooth horizontal panning in response to the defined PWM signals. After integrating the pushbutton, the system demonstrated correct input–output behavior, where the servo only activated when the button was pressed, confirming proper integration between user input and actuator control. Throughout the testing process, the system maintained stable operation, quick response times, and reliable performance without significant interruptions. Overall, the findings confirm that the ESP32-CAM, when combined with servo motor control and manual triggering via a pushbutton, can function as a simple yet effective smart surveillance system suitable for basic surveillance and monitoring applications.

8.0 Discussion

This experiment utilizes the ESP32-CAM module to enable live video streaming while simultaneously controlling a connected servo motor. The system operates over a shared Wi-Fi network, allowing users to access the video feed online. A built-in HTML interface is provided to simplify servo control, enabling users to remotely adjust its movement. This setup demonstrates how an embedded system can effectively integrate networking, real-time video streaming, and mechanical actuation within a compact platform.

In addition to manual control, the system incorporates a basic form of motion detection. The ESP32-CAM monitors changes in image brightness to identify whether the scene is static or in motion. When minimal or no changes are detected over several frames, the system automatically halts the servo movement and displays a message indicating that stillness has been

detected. This feature illustrates how simple image-processing techniques can be applied to camera input to trigger physical responses.

Furthermore, physical pushbuttons provide an alternative method for controlling the servo motor. When the button is pressed, the servo can be enabled or disabled, preventing any movement from being transmitted to it. In this state, even if the ESP32-CAM continues processing video, the servo remains inactive. Overall, this experiment demonstrates the integration of live video streaming, servo motor control, and basic motion detection within a single ESP32-CAM-based system.

9.0 Conclusion

This experiment successfully demonstrated the ESP32-CAM's ability to integrate video streaming, servo motor control, and basic motion detection into a working IoT system. The results show that the device can not only provide real-time camera feedback via a web interface but also intelligently respond to environmental changes by stopping the servo motor even when there is movement. The addition of a physical button further strengthens the usability of the system and offers an alternative way to control the servo. Overall, this experiment proves that the ESP32-CAM is a powerful and versatile platform for developing smart monitoring and automation applications.

10.0 Recommendations

It is important to remind users that after uploading or viewing the program, the IO0 pin must be disconnected from GND immediately. Failure to do so will prevent the ESP32-CAM from booting normally, as it will remain in programming mode. For future enhancements, the motion detection algorithm can be improved by adopting more advanced techniques such as pixel-by-pixel comparison or AI-based detection methods to achieve higher accuracy and reduce false triggers. Additionally, implementing password protection on the web interface is

recommended to enhance system security, particularly when operating on shared or public Wi-Fi networks.

Furthermore, integrating extra sensors such as PIR motion sensors or sound detection modules can significantly improve the system's reliability and responsiveness. These additions would help minimize false detections and make the overall system more robust and intelligent, resulting in a more advanced and practical surveillance solution.

11.0 References

- Use an ESP32-CAM to stream HD video

  https://core-electronics.com.au/guides/esp32-cam-set-up/

- How to Use Program and Upload Code to ESP-32 CAM

  https://randomnerdtutorials.com/program-upload-code-esp32-cam/

12. Acknowledgments

Special thanks to Dr. ZULKIFLI BIN ZAINAL ABIDIN and Dr. WAHJU SEDIONO for their guidance and support during this experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	Read	[/]
Matric Number: 2310663	Understand	[/]
Signature: SYAHMI	Agree	[/]

Name: ALYA SYAFINA BINTI ABDUL RAZAK	Read	[/]
Matric Number: 2218084	Understand	[/]
Signature: SYAF	Agree	[/]